Homework #2

In this assignment we will implement deep Q-learning, following DeepMind's paper ((2) and (1)) that learns to play Atari games. The purpose is to demonstrate the effectiveness of deep neural networks as well as some of the techniques used in practice to stabilize training and achieve better performance. In the process, you'll become familiar with pyTorch. We will train our networks on the *CartPole_Discrete* environment from pyRDDLGym, but the code can easily be applied to any other environment.

The CartPole environment corresponds to the version of the cart-pole problem described by Barto, Sutton, and Anderson in "*Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problem*". A pole is attached by an un-actuated joint to a cart, which moves along a frictionless track. The pendulum is placed upright on the cart and the goal is to balance the pole by applying forces in the left and right direction on the cart.

Since the goal is to keep the pole upright for as long as possible, by default, a reward of $+1$ is given for every step taken, including the termination step. The default reward threshold is 200 due to the time limit on the environment. This discrete version of the environment allows for two actions: (0) push the cart to the right (1) push the cart to the left.

The episode ends if any one of the following occurs:

1. Termination: Pole Angle is greater than $\pm 12^{\circ}$
2. Termination: Cart Position is greater than ± 2.4 (center of the cart reaches the edge of the display)
3. Truncation: Episode length is greater than 200.

Question 1. (Environment preparation)

The basic CarPole environment is quite naive. We will create first two variations for the purpose of this assignment. We will use the domain and instance code in this link: `https://github.com/pyrddlgym-project/rddlrepository/tree/main/rddlrepository/archive/gym/CartPole/Discrete` as a basis for our versions.

1. Update the actions space: First we will update the action space for this environment. Currently the environment allows for pushing left or right, however this is very inefficient and we wish to add "do nothing" action to prevent jittering. Augment the action space of the problem in the domain file. make sure to update the action-preconditions and the cpfs.

2. Now we will make sure we have two versions of the problem:

   (1) The first version is the one we already have.

   (2) For the second version we will change the reward function. Instead of just giving a $+1$ reward for every step the pole did not drop beyond the threshold, we will give a reward based on the actual values of the states

$$r_t = -pos_t^2 - pos\_ang_t^2 - ang_t^2 - ang\_vel_t^2 - 0.1 \cdot \left(\frac{force_t}{FORCE\_MAG}\right)^2$$

Question 2. (Q-learning)

Tabular setting If the state and action spaces are sufficiently small, we can simply maintain a table containing the value of $Q(s,a)$ – an estimate of $Q^*(s,a)$ – for every (s,a) pair. In this *tabular setting*, given an experience sample (s,a,r,s'), the update rule is

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left(r + \gamma \max_{a' \in \mathcal{A}} Q\left(s', a'\right) - Q\left(s,a\right) \right) \tag{1}$$

where $\alpha > 0$ is the learning rate, $\gamma \in [0, 1)$ the discount factor.

Approximation setting Due to the scale of continuous control environments, we cannot reasonably learn and store a Q value for each state-action tuple. We will instead represent our Q values as a function $\hat{q}(s, a; \mathbf{w})$ where $\mathbf{w}$ are parameters of the function (typically a neural network's weights and bias parameters). In this *approximation setting*, the update rule becomes

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left(r + \gamma \max_{a' \in \mathcal{A}} \hat{q}\left(s', a'; \mathbf{w}\right) - \hat{q}\left(s, a; \mathbf{w}\right) \right) \nabla_{\mathbf{w}} \hat{q}(s, a; \mathbf{w}). \tag{2}$$

In other words, we aim to minimize

$$L(\mathbf{w}) = \mathop{\mathbb{E}}_{s,a,r,s' \sim \mathcal{D}} \left[\left(r + \gamma \max_{a' \in \mathcal{A}} \hat{q}\left(s', a'; \mathbf{w}\right) - \hat{q}(s, a; \mathbf{w}) \right)^2 \right] \tag{3}$$

Target Network DeepMind's paper (2) (1) maintains two sets of parameters, $\mathbf{w}$ (to compute $\hat{q}(s, a)$) and $\mathbf{w}^-$ (target network, to compute $\hat{q}(s', a')$) such that our update rule becomes

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left(r + \gamma \max_{a' \in \mathcal{A}} \hat{q}\left(s', a'; \mathbf{w}^-\right) - \hat{q}\left(s, a; \mathbf{w}\right) \right) \nabla_{\mathbf{w}} \hat{q}(s, a; \mathbf{w}). \tag{4}$$

and the corresponding optimization objective becomes

$$L^-(\mathbf{w}) = \mathop{\mathbb{E}}_{s,a,r,s' \sim \mathcal{D}} \left[\left(r + \gamma \max_{a' \in \mathcal{A}} \hat{q}\left(s', a'; \mathbf{w}^-\right) - \hat{q}(s, a; \mathbf{w}) \right)^2 \right] \tag{5}$$

The target network's parameters are updated to match the Q-network's parameters every C training iterations, and are kept fixed between individual training updates.

Replay Memory As we interact, we store our transitions (s, a, r, s') in a buffer $\mathcal{D}$.

Old examples are deleted as we store new transitions. To update our parameters, we *sample* a minibatch from the buffer and perform a stochastic gradient descent update.

ϵ-Greedy Exploration Strategy For exploration, we use an ϵ-greedy strategy. This means that with probability ϵ, an action is chosen uniformly at random from $\mathcal{A}$, and with probability $1-\epsilon$, the greedy action (i.e., $\arg\max_{a\in\mathcal{A}} \hat{q}(s,a,\mathbf{w})$) is chosen. DeepMind's paper (2) (1) linearly anneals ϵ from 1 to 0.1 during the first million steps. At test time, we will set the agent to chooses the greedy action, i.e., $\epsilon_{\text{test}} = 0$.

There are several things to be noted:

- In this assignment, we will update $\mathbf{w}$ every learning_freq steps by using a minibatch of experiences sampled from the replay buffer.
- DeepMind's deep Q network takes as input the state s and outputs a vector of size $|\mathcal{A}|$. In the CartPole environment, we have $|\mathcal{A}| = 3$ actions, so $\hat{q}(s;\mathbf{w}) \in \mathbb{R}^3$.

We will now examine these assumptions and implement the ϵ-greedy strategy.

1. (**written**) What is one benefit of representing the Q function as $\hat{q}(s;\mathbf{w}) \in \mathbb{R}^{|\mathcal{A}|}$?

We will now investigate some of the theoretical considerations involved in the tuning of the hyperparameter C which determines the frequency with which the target network weights $\mathbf{w}^-$ are updated to match the Q-network weights $\mathbf{w}$. On one extreme, the target network could be updated *every* time the Q-network is updated; it's straightforward to check that this reduces to not using a target network at all. On the other extreme, the target network could remain fixed throughout the entirety of training.

Furthermore, recall that stochastic gradient descent minimizes an objective of the form $J(\mathbf{w}) = \mathbb{E}_{x\sim\mathcal{D}}[l(x, \mathbf{w})]$ by making sample updates of the following form:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla_{\mathbf{w}} l(x, \mathbf{w})$$

Stochastic gradient descent has many desirable theoretical properties; in particular, under mild assumptions, it is known to converge to a local optimum. In the following questions we will explore the conditions under which Q-Learning constitutes a stochastic gradient descent update.

2. (**written**) Consider the first of these two extremes: standard Q-Learning without a target network, whose weight update is given by equation **(2)** above. Is this weight update an instance of stochastic gradient descent (up to a constant factor of 2) on the objective $L(\mathbf{w})$ given by equation **(3)**? Argue mathematically why or why not.

3. (**written**) Now consider the second of these two extremes: using a target network that is never updated (i.e. held fixed throughout training). In this case, the weight update is given by equation **(4)** above, treating $\mathbf{w}^-$ as a constant. Is this weight update an instance of stochastic gradient descent (up to a constant factor of 2) on the objective $L^-(\mathbf{w})$ given by equation **(5)**? Argue mathematically why or why not.

Implementing DeepMind's DQN

4. (**coding**) Implement the deep Q-network as described in (2). Remember to implement all the components described above. Create a function *sample_action(Explore=Tr* which gets an argument that toggles the exploration-exploitation scheme (False means pure exploitation).

5. (**coding**) Run your implementation of DQN for both versions of CartPole you created for 10K steps. For each report the episode's accumulated reward as a

function of the iteration. In practice, every 10 episodes, run 10 evaluation (pure exploitation) episodes, plot the mean and an envelope of the std around the mean (the output plot should have 1000 evaluation points).

*Make sure to start each episode (training or evaluation) from a random feasible state.

Note:

(1) Make sure to start each episode (training or evaluation) from a random feasible state

(2) It is left for your discrete to decide on the neural network architecture, but please restrict yourselves to 2-layers MLPs.

6. (**written**) Attach the plots of rewards. What you say about the time it takes to converge in each scenario (the reward scales are different, please refer to the trend).

References

[1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, Martin Riedmiller Playing atari with deep reinforcement learning *NIPS Deep Learning Workshop*, 2013.

[2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.